

Finite Automata in Haskell

V. Velkey, W. Vromen

June 1, 2023

Overview

- ▶ Theory overview: DFA-s, NFA-s, ϵ -NFA-s, Regular expressions
- ▶ DFA -s: implementation, evaluation of words
- ▶ NFA-s and ϵ -NFA-s: implementation, evaluation
- ▶ Powerset construction: ϵ -NFA to DFA
- ▶ RegExp: implementation, word generation
- ▶ DFA to RegExp
- ▶ RegExp to ϵ -NF
- ▶ Tests

DFA -s: implementation, evaluation of words

Design choice: words with unknown symbols will be *rejected*.

```
type Symbol = Char
data DFA a = DFA { states :: [a]
    , alphabet :: [Symbol]
    , delta :: Symbol -> a -> a
    , start :: a
    , acceptstate :: [a]}

evaluate :: (Eq a, Ord a) => DFA a -> String -> Bool
evaluate df st = all ('elem' alphabet df) st && -- all characters in
    string are in the alphabet
    stateArr (start df) st 'elem' acceptstate df where
        stateArr q [] = q
        stateArr q (x:xs) = stateArr (delta df x q) xs
```

NFA implementation

Transitioning between lists of states: double recursion.

```
data NFA a = NFA { statesNF :: [a]
                  , alphabetNF :: [Symbol]
                  , deltaNF :: Symbol -> a -> [a]
                  , startNF :: a
                  , acceptstateNF :: [a]}

evaluateNF :: Eq a => NFA a -> String -> Bool
evaluateNF nf st = all ('elem' alphabetNF nf) st && -- captures elements
  of string in alphabet
    any ('elem' acceptstateNF nf) (stateArrNF' [startNF nf] st)
      where
        stateArrNF' qs [] = qs
        stateArrNF' [] _ = []
        stateArrNF' [q] (x:xs) = stateArrNF' (deltaNF nf x q)
          xs --recursion on string
        stateArrNF' (q:qs) (x:xs) = stateArrNF' [q] (x : xs) '
          union' stateArrNF' qs (x : xs) --recursion on
          statespace
```

RegExp implementation: data type

```
data RegExp = Empty | Epsilon | R [Symbol] | Union RegExp RegExp
            | Star RegExp | Con RegExp RegExp | Plus RegExp
  deriving (Show, Eq)
```

Design choices:

- ▶ not split empty from non-empty expressions
- ▶ Epsilon versus R []
- ▶ R [Symbol] versus R Symbol

For convenience:

```
regExpUnion :: [RegExp] -> RegExp
regExpUnion = simplify . foldr Union Empty
```

RegExp but prettier

```
pRegExp :: RegExp -> String
pRegExp Empty = ""
pRegExp Epsilon = "R e"
pRegExp (R []) = "R e"
pRegExp (R xs) = show xs
pRegExp (Union r1 r2) = "(" ++ pRegExp r1 ++ "|" ++ pRegExp r2 ++ ")"
pRegExp (Star r) = "(" ++ pRegExp r ++ ")*"
pRegExp (Con r1 r2) = pRegExp r1 ++ pRegExp r2
pRegExp (Plus r) = "(" ++ pRegExp r ++ "+)"

ppRegExp :: RegExp -> IO ()
ppRegExp = print . pRegExp
```

ghci> ppRegExp (Con (Union (R "53") (R " ")) (Star (R "19"))))

then becomes

("("53"|R e)("19")*)"

RegExp Arbitrary

```
instance Arbitrary RegExp where
  arbitrary = sized randomReg where
    randomReg :: Int -> Gen RegExp
    randomReg 0 = R <$> elements ["0", "1"]
    randomReg n = oneof [ R <$> elements ["0", "1"]
      , Star <$> randomReg (n `div` 8)
      , Plus <$> randomReg (n `div` 8)
      , Con <$> randomReg (n `div` 8)
      , <*> randomReg (n `div` 8)
      , Union <$> randomReg (n `div` 8)
      , <*> randomReg (n `div` 8)
      , return Epsilon ]
```

We chose 'div' 8 to avoid dealing with monstrosities

RegExp to Generate Words

```
generateString :: RegExp -> Gen String
generateString = ('generateString', 5) . simplify
  where generateString' :: RegExp -> Int -> Gen String
        generateString' Empty _ = error "cannot generate from empty
            language"
        generateString' Epsilon _ = return ""
        generateString' (R xs) _ = return xs
        generateString' (Union r1 r2) n = oneof [generateString' r1 n,
            generateString' r2 n]
        generateString' (Con r1 r2) n = do
            w <- generateString' r1 n
            v <- generateString' r2 n
            return (w ++ v)
        generateString' (Star _) 0 = return ""
        generateString' (Star r) n = oneof [generateString' Epsilon n,
            generateString' (Con r (Star r)) (n - 1)]
        generateString' (Plus r) n = generateString' (Con r (Star r)) n
```

Design choice: ϵ -transitions represented as list of tuple - ease with transitive closure

```
data ENFA a = ENFA { statesENF :: [a]
    , alphabetENF :: [Symbol]
    , deltaENF :: Symbol -> a -> [a]
    , epTrans :: [(a, a)]
    , startENF :: a
    , acceptstateENF :: [a] }

rtClose :: (Eq a, Ord a) => ENFA a -> [a] -> [a]
rtClose nf qs = sort (unionL [[p | p <- statesENF nf , (q, p) 'elem'
    trClose rcl ] | q <- qs] ) where
    rcl = refClose (statesENF nf) (epTrans nf)
```

The powerset construction - definition

- ▶ Take NFA $N = (Q_n, \Sigma, \delta_n, q_0, F_n)$
- ▶ Powerset construct: $D = (\mathcal{P}(Q_n), \Sigma, \delta, S, F_D)$
 - ▶ States: subsets of Q_n
 - ▶ Alphabet: same (of course)
 - ▶ transition function: $\delta_D(R, \sigma) = \cup_{s \in R} \delta_N(s, \sigma)$
 - ▶ Start state: singleton with previous start state
 - ▶ Accept states: $\{R \subset Q_N : S \cap F_n \neq \emptyset\}$
- ▶ For ϵ -NFA:
Similar idea, but at each step, need to take ϵ -closure at steps
 - ▶ Start state $\text{ecl}(\{q_0\})$
 - ▶ transition function: $\delta_D(R, \sigma) = \text{ecl}(\cup_{s \in R} \delta_N(s, \sigma))$